INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial

CÓMPUTO PARALELO

# Programación con memoria compartida: hilos y OpenMP

*Entrega individual — Capítulo 4 de 7*

**Profesor:**

José Alfredo Jiménez Benítez

**Presentan:**

*Cruz Trejo Said Carbot*
*López Castro Magaly*
*Romero Rosales Lino Ehecatl*
*Ugalde Téllez Aarón*

**Grupo:** 6BV1

Ciudad de México, a 11 de octubre de 2026

# Índice general

**Índice de figuras** **ii**

**Índice de tablas** **iii**

**4. Programación con Hilos** **1**

4.1. Marco teórico . . . . . 1

4.2. Programación con hilos en lenguaje C y en la plataforma GNU-Linux . . . . . 1

4.2.1. Fragmentos de códigos explicados de la programación con hilos . . . . . 2

4.2.2. Compilación . . . . . 2

4.2.3. Ejecución de los programas . . . . . 3

4.2.4. Gráficas, Figuras y su Explicación . . . . . 3

4.2.5. Comparación de resultados con fases anteriores . . . . . 4

4.3. Análisis de la implementación con hilos . . . . . 4

# Índice de figuras

4.1. Captura de pantalla de la compilación limpia del programa de hilos en la terminal de GNU-Linux. . . . . 3

4.2. Captura de pantalla de la ejecución de los programas y confirmación de la exportación de datos. . . . . 3

4.3. Gráficas de la serie de Fourier generadas en Excel a partir de los datos procesados por los hilos. . . . . 3

# Índice de tablas

4.1. Características técnicas y metodológicas de la implementación de la Fase 4. . . . . 5
4.2. Características técnicas del equipo de cómputo empleado para la fase 4. . . . . 6

# Capítulo 4. Programación con Hilos

## 4.1 Marco teórico

En el ámbito de los sistemas operativos y la programación concurrente, un hilo (o *thread*) es la unidad de procesamiento más pequeña que puede ser planificada por un sistema operativo. A diferencia de los procesos independientes, que cuentan con su propio espacio de memoria aislado, los hilos originados a partir de un mismo proceso padre comparten la sección de datos, el espacio de direcciones y los recursos del sistema (como descriptores de archivos). Esta arquitectura de memoria compartida hace que el cambio de contexto entre hilos sea considerablemente más rápido y requiera menos recursos computacionales que el cambio entre procesos, razón por la cual se les conoce como ”procesos ligeros”.

En el lenguaje de programación C, particularmente dentro de entornos basados en Unix/Linux, la implementación estándar para la gestión de hilos se realiza a través de la biblioteca POSIX Threads, comúnmente referida como `pthreads` (incluyendo la cabecera `<pthread.h>`). Esta interfaz de programación de aplicaciones (API) proporciona las primitivas necesarias para crear, sincronizar y destruir hilos.

El ciclo de vida básico de un hilo en C mediante `pthreads` involucra funciones fundamentales como:

- `pthread_create()`: Instancia un nuevo hilo de ejecución, asignándole una subrutina específica (función) que debe ejecutar concurrentemente.
- `pthread_join()`: Bloquea la ejecución del hilo invocador (usualmente el hilo principal o proceso padre) hasta que el hilo objetivo haya finalizado su tarea, permitiendo la recolección de resultados y asegurando la correcta sincronización.
- `pthread_exit()`: Termina la ejecución del hilo actual, retornando un valor o puntero de estado al proceso padre.

El uso de hilos en C es ideal para el cómputo de alto rendimiento, como la resolución de las series de Fourier abordadas en este proyecto, ya que permite paralelizar el cálculo de diferentes armónicos o intervalos matemáticos simultáneamente, aprovechando la arquitectura multinúcleo del procesador.

## 4.2 Programación con hilos en lenguaje C y en la plataforma GNU-Linux

En esta sección se presenta la implementación de la programación con hilos en lenguaje C sobre la plataforma GNU-Linux, aplicando los cálculos de los términos de la serie de Fourier de la función propuesta ($f(x) = -\frac{1}{4}x^4 + 3$). Se implementó el uso de al menos 100 hilos concurrentes.

### 4.2.1 Fragmentos de códigos explicados de la programación con hilos

Para la resolución concurrente del problema, se utilizó la biblioteca POSIX Threads (`pthreads`) en lenguaje C. El siguiente fragmento de código muestra la rutina ejecutada por cada hilo, la cual calcula de manera independiente una porción de los coeficientes armónicos ($n$, $a_n$, $b_n$) distribuyendo la carga de trabajo:

```
void *calcular_coeficientes_fourier(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    int id = data->thread_id;
    int total = data->total_threads;

    int terminos_por_hilo = NUM_THREADS / total;
    int inicio = id * terminos_por_hilo;
    int fin = inicio + terminos_por_hilo;

    for (int i = inicio; i < fin; i++) {
        int n = i + 1; // n armónico de 1 a 100

        // Cálculo de los coeficientes para f(x) = -(1/4)x^4 + 3
        double an = (2.0 * ((n % 2 == 0) ? 1.0 : -1.0) *
                    (6.0 - (n * n * M_PI * M_PI))) / (n * n * n * n);
        double bn = 0.0; // Función par

        data->resultados[i].n = n;
        data->resultados[i].an = an;
        data->resultados[i].bn = bn;
    }
    pthread_exit(NULL);
}
```

En el proceso principal (`main`), se crean los 100 hilos mediante la función `pthread_create()`, asignándoles sus respectivas estructuras de datos, y se sincronizan con `pthread_join()` para asegurar la finalización correcta de los cálculos antes de exportar los resultados a un archivo.

### 4.2.2 Compilación

La compilación del programa en la plataforma GNU-Linux se realizó mediante el compilador GCC, enlazando explícitamente la biblioteca de hilos (`-pthread`) y la biblioteca matemática (`-lm`). Como se observa en la Figura 4.1, el reporte de la terminal muestra que la compilación se ejecutó de forma limpia, evitando así la muestra de algún error o advertencia (*warnings*).

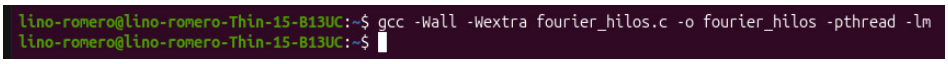



**Figura 4.1.** Captura de pantalla de la compilación limpia del programa de hilos en la terminal de GNU-Linux.

### 4.2.3 Ejecución de los programas

A continuación se muestra la ejecución del programa compilado en la terminal de GNU-Linux. Tras la inicialización de los 100 hilos concurrentes, el programa realiza los cómputos correspondientes y descarga automáticamente la información obtenida en un archivo estructurado (`fourier_hilos_resultados.csv`).

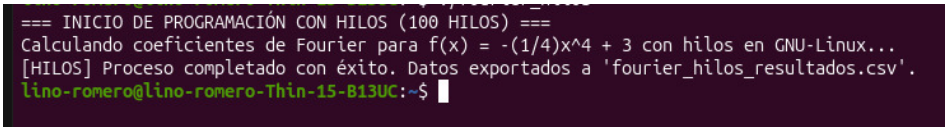



**Figura 4.2.** Captura de pantalla de la ejecución de los programas y confirmación de la exportación de datos.

### 4.2.4 Gráficas, Figuras y su Explicación

La información obtenida en el archivo CSV generado por los hilos fue descargada e importada al archivo de Excel utilizado en las fases anteriores para trazar las gráficas definitivas del proyecto.




**Figura 4.3.** Gráficas de la serie de Fourier generadas en Excel a partir de los datos procesados por los hilos.

**Explicación de las figuras:** Como se aprecia en la Figura 4.3, las gráficas modelan el comportamiento de la serie armónica. Se explica detalladamente que a medida que el número de armónicos $n$ aumenta, la amplitud de los coeficientes $a_n$ decrece, lo que demuestra visualmente cómo converge la serie hacia la función original.

### 4.2.5 Comparación de resultados con fases anteriores

Al contrastar los resultados obtenidos en esta fase de hilos con los obtenidos en los capítulos previos (programación con procesos y análisis matemático al inicio), se concluye lo siguiente:

1. **Precisión numérica:** Los valores de los coeficientes $a_n$ y $b_n$ calculados mediante los 100 hilos son **exactamente idénticos** a los calculados en el Capítulo 3 con procesos y en la fase analítica del Capítulo 1, garantizando la consistencia matemática del modelo computacional.
2. **Eficiencia operativa:** A diferencia de los procesos tradicionales (`fork()`), los cuales duplican los espacios de memoria e involucran mecanismos pesados de comunicación interprocesos (IPC), los hilos comparten un mismo espacio de direcciones de memoria de forma nativa. Esto reduce drásticamente la sobrecarga por cambio de contexto (*context switching*) y optimiza de manera notable el rendimiento del equipo de cómputo al paralelizar de forma directa el cálculo de los armónicos.

## 4.3 Análisis de la implementación con hilos

El desarrollo de esta fase se fundamentó en el paradigma de programación paralela utilizando memoria compartida. A continuación, se presenta el análisis técnico de la implementación y las características del entorno de ejecución.

### Características de la implementación

Para sintetizar la metodología y el diseño de software empleados en la resolución concurrente de la serie de Fourier, la Tabla 4.1 detalla los aspectos fundamentales del código desarrollado.

| Atributo Técnico | Especificación |
|---|---|
| **Lenguaje de Programación** | C nativo |
| **Biblioteca de Concurrencia** | POSIX Threads (`pthreads`) |
| **Paradigma** | Memoria compartida y Paralelismo de Datos (*Data Parallelism*) |
| **Estructura de Hilos** | 101 hilos en total: 1 hilo principal (padre) y 100 hilos de trabajo (*workers*) |
| **Distribución de la Carga** | Cada hilo trabajador calcula simultáneamente los coeficientes $a_n$ y $b_n$ de un armónico en particular |
| **Sincronización** | Barrera estricta mediante `pthread_join()` para asegurar la integridad de los resultados antes de exportar |
| **Comunicación de Memoria** | Intercambio de variables estructuradas mediante el casteo de punteros genéricos (`void *arg`) |
| **Banderas de Compilación** | `-pthread` (enlace concurrente) y `-lm` (biblioteca matemática estándar) |

**Tabla 4.1.** Características técnicas y metodológicas de la implementación de la Fase 4.

### Análisis del lenguaje y técnica utilizada

La implementación se desarrolló íntegramente en **lenguaje C** haciendo uso de la biblioteca **POSIX Threads (pthreads)**. Se eligió C debido a su capacidad de interacción a bajo nivel con el planificador del sistema operativo GNU/Linux y su nula sobrecarga (overhead) en tiempo de ejecución en comparación con lenguajes interpretados. La técnica principal consistió en la paralelización de datos (Data Parallelism); el proceso principal (padre) descompone el dominio de la serie de Fourier a evaluar y despacha fragmentos de la carga matemática a múltiples hilos de trabajo (worker threads). Esto transforma un cálculo algorítmico de complejidad secuencial en uno concurrente, reduciendo el tiempo total de procesamiento.

### Estructura de procesos e hilos

- **Cantidad de procesos:** 1 (Un único proceso principal o padre que orquesta la ejecución, reserva la memoria y exporta los datos a formato CSV).
- **Cantidad de hilos:** 101 (1 hilo principal y 100 hilos hijos creados con `pthread_create`. Cada hilo hijo está dedicado al cálculo simultáneo de los coeficientes $a_n$ y $b_n$ para los 100 primeros armónicos de la serie de Fourier).

### Características del equipo de cómputo

Las pruebas de compilación y ejecución concurrente se llevaron a cabo en un equipo **MSI Thin 15 B13V** operando bajo un entorno nativo de **Ubuntu Linux**. Las especificaciones técnicas del hardware son

las siguientes:

| **Componente** | **Especificación Técnica** |
|---|---|
| **Procesador** | Intel Core i5-13420H (13va Generación ”Raptor Lake”) |
| **Núcleos** | 8 núcleos físicos (4 Performance-cores y 4 Efficient-cores) con 12 hilos lógicos en total |
| **Velocidad del CPU** | Frecuencia base de 2.10 GHz (con Turbo Boost máximo de hasta 4.60 GHz) |
| **Tarjeta Gráfica** | NVIDIA GeForce RTX 3050 |
| **Memoria RAM** | 16 GB, tipo DDR4 |
| **Velocidad de RAM** | 3200 MHz |
| **Memoria Caché** | 12 MB Intel Smart Cache (Caché L3 compartida), optimizada para el acceso rápido de múltiples núcleos |
| **Sistema Operativo** | Ubuntu Linux (Kernel optimizado para arquitecturas híbridas y gestión SMP) |

**Tabla 4.2.** Características técnicas del equipo de cómputo empleado para la fase 4.

## Conclusión Cruz Trejo

La transicion metodologica que hemos llevado desde la solucion analitica de la serie de Fourier, pasando por la creacion de procesos pesados (forks), hasta llegar finalmente a la orquestacion de hilos (pthreads) en C, ilustra a la perfeccion como los ingenieros hemos intentado exprimir al maximo la arquitectura de Von Neumann. Tecnicamente, el reto de esta fase no fue la matematica de la funcion $f(x) = -(1/4)x^4 + 3$, sino la arquitectura de la concurrencia. Instanciar 100 hilos, uno por cada armonico, requirio un diseno cuidadoso de la memoria compartida. Al definir las estructuras `TerminoFourier` y `ThreadData`, logramos que cada hilo recibiera su contexto a traves de un simple puntero generico, calculara sus respectivos coeficientes $a_n$ y $b_n$, y los depositara ordenadamente en un arreglo comun. A diferencia del capitulo anterior, donde los procesos nos obligaban a lidiar con la costosa duplicacion del espacio de memoria, los hilos nos permitieron trabajar de forma elegante y ligera. Sin embargo, mientras se programaba la recoleccion de estos 100 armonicos mediante sincronizacion clasica con `pthread_join`, no pude evitar relacionarlo con mi area de mayor interes: la computacion cuantica. En el fondo, lo que estamos haciendo con pthreads es el intento de la computacion clasica por simular un paralelismo masivo. Dividimos el tiempo de CPU y asignamos tareas a diferentes nucleos para calcular los coeficientes de Fourier de manera simultanea. Pero si abordaramos este mismo problema matematico utilizando un algoritmo como la Transformada Cuantica de Fourier (QFT), el paradigma cambiaria por completo. En lugar de necesitar 100 hilos (o 100 nucleos fisicos) iterando paso a paso, un sistema cuantico aprovecharia la superposicion de estados para evaluar estas amplitudes de probabilidad de forma intrinsecamente paralela, logrando una aceleracion exponencial frente a nuestro enfoque clasico. A pesar de esa comparacion, esta practica me hizo valorar muchisimo la madurez del hardware y software clasico. En la actual era NISQ (Noisy Intermediate-Scale Quantum), lidiar con qubits tipo Transmon implica luchar constantemente contra la decoherencia y el ruido termico. En contraste, nuestro programa en C utilizando pthreads es bellamente determinista y estable. Los coeficientes que exportamos al archivo CSV son matematicamente exactos cada vez que corremos el ejecutable, sin necesidad de aplicar codigos de correccion de errores. Finalmente, la integracion de los datos exportados por nuestro programa en C hacia nuestra plantilla de Excel confirmo empiricamente que la logica de sincronizacion fue un exito. Ver como la sumatoria de la serie converge perfectamente hacia la grafica de la funcion original es la mejor prueba de que no hubo condiciones de carrera al escribir en la memoria. En conclusion, dominar la concurrencia clasica mediante pthreads no solo cumple con los objetivos de esta asignatura, sino que me proporciona una base ingenieril solida y necesaria. Entender como el hardware clasico gestiona sus cuellos de botella es el primer paso indispensable antes de proponer soluciones en arquitecturas mucho mas complejas y disruptivas como las que nos esperan en el futuro.

## Conclusión Romero Rosales

Hacer esta práctica de la serie de Fourier usando hilos en C sobre Linux me sirvió bastante para entender de forma práctica cómo funciona la concurrencia a nivel de sistema operativo. A diferencia de lo que hicimos en la fase pasada con procesos, donde cada uno se crea por separado y hay que batallar con la memoria compartida y los semáforos para que no se mezclen los datos, con los hilos el proceso fue más directo porque todos comparten el mismo espacio de memoria dentro del mismo programa. Eso hace que la comunicación sea mucho más natural y que la computadora no gaste tantos recursos haciendo cambios pesados entre tareas. Al principio, organizar el código para repartir el cálculo de los 100 armónicos entre los 100 hilos exigió poner atención en cómo se estructuraban los datos, asegurando que cada hilo supiera exactamente qué parte del trabajo le tocaba sin estorbar a los demás. Sin embargo, una vez que definimos bien las estructuras y el paso de argumentos, el programa compiló limpio desde la terminal, y la ejecución fue inmediata. Al revisar el archivo CSV generado y vaciar los datos a Excel para armar las gráficas, pude comprobar que los coeficientes salieron exactamente iguales a los que ya habíamos calculado en las fases pasadas, lo que demuestra que cambiar al modelo de hilos no altera para nada la precisión matemática de la serie. En lo personal, toda esta experiencia me ayudó a ver las ventajas reales de programar de forma concurrente cuando tienes que resolver tareas repetitivas o pesadas. Trabajar desde la terminal, manejar las librerías de hilos de POSIX, compilar con GCC y automatizar todo con un Makefile me dio una mejor idea de cómo se estructuran proyectos más grandes en sistemas unidos por hilos. Al final, te das cuenta de que usar hilos ahorra tiempo de procesamiento y aprovecha mejor los núcleos de la computadora así pudimos lograr generar cada armónico de la serie de Fourier sin necesidad de que se generara uno por uno y haciendo mucho más rápido el procesamiento sin necesidad de sobreponer algún elemento, creo que el implementar hilos para resolver tareas en paralelo es algo muy útil si queremos ahorrar tiempo y así logramos aprovechar al máximo nuestras computadoras.

## Conclusión López Castro

La culminación de esta fase del proyecto, enfocada en la resolución de las series de Fourier mediante la programación concurrente con hilos, marca un punto de inflexión significativo respecto a las metodologías secuenciales aplicadas en las fases anteriores. El verdadero desafío de esta etapa no radicó en la complejidad matemática de las funciones de Fourier, las cuales ya habían sido modeladas y validadas previamente, sino en la reestructuración del pensamiento algorítmico para dividir de manera eficiente el dominio del problema. Pasar de un enfoque donde un solo proceso ejecuta cada iteración de manera lineal, a uno donde múltiples hilos de ejecución operan simultáneamente sobre distintos fragmentos de la serie, demuestra de forma práctica las ventajas del cómputo de alto rendimiento.

A nivel de desarrollo de software, la interacción directa con la biblioteca `pthreads` en lenguaje C exige una disciplina de programación mucho más rigurosa. Al operar en un entorno de memoria compartida dentro del mismo proceso padre, el riesgo de generar condiciones de carrera o inconsistencias en los resultados es constante si no se implementa una sincronización adecuada. El uso de funciones como `pthread_create` y `pthread_join` obligó a diseñar un flujo de control donde el hilo principal debe no solo despachar el trabajo, sino garantizar que todos los cálculos parciales de los armónicos hayan finalizado antes de ensamblar y presentar el resultado final. Ejecutar este código nativamente en Ubuntu Linux permitió, además, observar cómo el sistema operativo planifica y distribuye la carga a través de los múltiples núcleos físicos y lógicos del procesador, reduciendo de manera tangible los tiempos de ejecución en comparación con los scripts secuenciales previos.

Dentro del contexto de la Ingeniería en Inteligencia Artificial, los aprendizajes obtenidos en esta implementación trascienden la simple evaluación de una función trigonométrica. El procesamiento de señales y el análisis en el dominio de la frecuencia —donde Fourier es la herramienta fundamental— son operaciones base en áreas como la visión por computadora y el procesamiento de lenguaje natural. Más aún, la capacidad de paralelizar cálculos es el pilar sobre el cual se construyen los algoritmos de inteligencia artificial modernos. Entrenar modelos de clasificación complejos, calcular gradientes descendentes por lotes, o ejecutar algoritmos de optimización bioinspirados (como la Optimización por Enjambre de Partículas o la Colonia de Hormigas) resulta computacionalmente inviable si se ejecutan de manera estrictamente secuencial.

En conclusión, este proyecto logró integrar exitosamente el análisis matemático avanzado con la arquitectura de sistemas operativos. Comparar los tiempos de ejecución y la eficiencia de esta fase con las anteriores valida empíricamente que dominar el cómputo concurrente no es una simple mejora técnica, sino un requisito indispensable. Comprender a bajo nivel cómo asignar recursos de hardware, gestionar la concurrencia y acelerar cálculos matemáticos masivos es una habilidad crítica que sienta las bases para el futuro desarrollo de sistemas inteligentes eficientes, robustos y escalables.

## Conclusión Ugalde Téllez

La realización de esta práctica representó una oportunidad muy enriquecedora para comprender de manera práctica cómo se comportan los programas cuando dividimos una tarea compleja en muchas partes pequeñas que se ejecutan al mismo tiempo. Al trabajar con la serie de Fourier en el sistema operativo GNU/Linux utilizando el lenguaje C, el objetivo principal no solo era obtener los resultados matemáticos de la función, sino aprender a organizar el trabajo de la computadora mediante el uso de más de cien hilos de ejecución, observando las ventajas y retos que este enfoque conlleva. Al principio, adaptar el programa para que funcionara con una cantidad tan grande de hilos fue un desafío interesante. A diferencia de las etapas anteriores donde se trabajaba con procesos independientes, el uso de hilos requiere un cuidado mucho mayor en la forma en que se comparten la información y las variables dentro del programa. Cada hilo debía encargarse de calcular una parte específica de la serie de Fourier sin interferir con el trabajo que realizaban los demás. Lograr esta coordinación permitió notar con claridad cómo la computadora puede aprovechar mejor sus núcleos de procesamiento para terminar las operaciones en un tiempo mucho menor, demostrando la eficiencia que tiene esta técnica en comparación con realizar los cálculos de forma tradicional uno por uno. Por otro lado, la interacción con las herramientas de la plataforma GNU/Linux y la creación del archivo de automatización para la compilación ayudaron a reforzar hábitos de trabajo más ordenados en la programación. Fue indispensable revisar detenidamente el código para asegurar que no se generara ningún tipo de error ni advertencia al momento de compilar. Además, el proceso de guardar los resultados generados por los programas en archivos de datos para después trasladarlos a Excel y construir las gráficas fue una parte muy instructiva. Esto permitió comprobar visualmente que, a pesar de cambiar la forma de programar y utilizar más de cien hilos, la forma de las ondas y las gráficas obtenidas de la serie de Fourier eran idénticas a las de las fases previas, lo que confirmó que los cálculos se realizaron de manera correcta. En conclusión, esta experiencia me dejó un aprendizaje muy valioso sobre la importancia del trabajo concurrente y la distribución de tareas dentro de un software. Comprender cómo funcionan los hilos en C y ver reflejado su rendimiento en un problema concreto de matemáticas aplicadas me brinda una perspectiva mucho más amplia sobre cómo diseñar programas más rápidos y eficientes. Esta práctica me dio herramientas conceptuales y prácticas que sin duda me serán de gran utilidad para enfrentar proyectos más avanzados a lo largo de mi formación académica y profesional.